



// AI RELIABILITY · SELF-ASSESSMENT

The 50-Point AI Reliability Audit

A working checklist for teams shipping LLM-powered features — agents, RAG, copilots — who want to know where their reliability gaps actually are before a customer finds them first.

How to use this

This isn't a certification and it isn't exhaustive — it's a working tool. Fifty concrete, checkable practices across eight areas that consistently separate AI systems that fail loudly in a demo from ones that fail quietly in production, six weeks after everyone stopped watching them closely.

Go through each section with your actual system in mind, not the system you're planning to build eventually. Check only what's genuinely true today. The value here isn't the score — it's seeing exactly which unchecked boxes cluster together, because that cluster is usually where your next real incident is going to come from.

01

Be honest, not aspirational

"We have a plan to add this" is unchecked. Only check what's built, tested, and running today.

02

Do it with your team

One person's view of "we handle that" is often wrong. A second set of eyes catches assumed coverage.

03

Revisit quarterly

Reliability posture drifts as fast as the system does. A one-time audit goes stale within a release cycle.

Reading your score

40–50 checked	Strong posture. Gaps left are likely edge cases worth a deliberate decision, not neglect.
25–39 checked	Workable, but with real exposure. Look for whether unchecked items cluster in one section — that's your priority.
10–24 checked	Meaningful gaps in core reliability practice. Worth a dedicated pass before the next major feature ships.
Under 10 checked	Most AI reliability work is still ahead of you. Not unusual for a pre-launch or early-stage system — the point is knowing it now.

01 Evaluation & Testing

Whether you actually know if your system's outputs are correct — not whether they look plausible.

☐ **You have a labeled eval set, not just anecdotal examples.**

A collection of real or representative inputs with known-good expected behavior, versioned like code.

☐ **Eval runs are part of your CI pipeline, not a manual pre-launch ritual.**

If evals only run when someone remembers to run them, they will eventually not run.

☐ **You test against adversarial and edge-case inputs, not just happy-path examples.**

Malformed input, contradictory context, and off-topic requests all belong in your eval set.

☐ **LLM-as-judge scoring, where used, has been validated against human judgment.**

A judge model that hasn't been checked against real human ratings is an unverified assumption, not a metric.

☐ **You have regression tests for previously-fixed failure modes.**

A bug that was fixed once and never turned into a test case is a bug that will recur silently.

☐ **Eval scores are tracked over time, not just checked at a single point.**

A model or prompt change that quietly degrades quality months later is invisible without a trend line.

☐ **Someone other than the system's builder has reviewed the eval set for blind spots.**

The person who built the system is the worst-positioned person to notice what it's not being tested for.

02 Guardrails & Safety

What stops a bad output from reaching a user, and whether that mechanism has actually been tested to fail.

- ☐ **There's a scoring or gating step between model output and user, not a direct pass-through.**
Even a lightweight check beats none — the question is whether anything sits in the path at all.
- ☐ **Prompt-injection resistance has been explicitly tested, not assumed.**
Test both whether an injection attempt is detected, and whether the model's response actually complied with it.
- ☐ **You've defined what happens on a guardrail failure — fail open or fail closed — deliberately, per use case.**
"We haven't decided" is itself a decision, and usually the wrong one.
- ☐ **Guardrail latency budgets are defined and enforced with timeouts.**
An unbounded safety check that occasionally hangs is a production outage waiting for the right input.
- ☐ **High-confidence blocks don't require a model call to resolve.**
If every single check round-trips to a judge model, cost and latency scale with traffic in a way that doesn't survive growth.
- ☐ **Guardrail logic has its own tests, independent of the feature it's protecting.**
A safety layer without direct test coverage can silently break while the feature it guards keeps working.

03 Grounding & Retrieval

Whether what your system says is actually supported by what it retrieved — the specific place fabrication hides.

- ☐ **There's an automated check for whether a response's claims are supported by its retrieved context.**

Not a spot-check by a human reviewer — an automated pass that runs on every response.

- ☐ **Your citation-checking logic accounts for negation, not just substring presence.**

A document stating a fact doesn't exist still contains that fact as a literal string — a naive check gets this backwards.

- ☐ **Retrieval quality is measured separately from generation quality.**

A good answer built on bad retrieval is luck. Knowing which stage failed is what makes a fix possible.

- ☐ **You test what happens when retrieval returns nothing relevant.**

A system that confidently answers anyway, with no relevant context, is a system built to hallucinate under load.

- ☐ **Stale or outdated source documents are detected, not silently served.**

An index that's never revalidated becomes a source of confidently wrong answers as the underlying data ages.

- ☐ **Embedding or index changes trigger a re-run of your retrieval evals.**

A model or chunking-strategy change can silently shift what gets retrieved without anyone noticing until a user does.

04 Prompt & Model Management

Whether prompt and model changes are treated with the same rigor as code changes — because they are code.

- ☐ **Prompts are version-controlled, not edited live in a dashboard.**

If you can't diff a prompt change or roll it back, you can't debug a regression it caused.

- ☐ **A prompt change requires the same review process as a code change.**

A one-line prompt edit can change behavior as much as a significant code change — treat it accordingly.

- ☐ **Model version pinning is deliberate, and upgrades are tested before rollout.**

An auto-upgrading model dependency is a silent behavior change waiting to happen on someone else's schedule.

- ☐ **You've tested behavior under a model provider outage or degradation.**

Fallback behavior — a cached response, a different provider, a graceful error — should be tested, not assumed.

- ☐ **Temperature and other generation parameters are set deliberately per use case, not left at defaults everywhere.**

The same settings rarely fit both a creative-writing feature and a factual-lookup feature.

- ☐ **There's a documented rollback plan for a bad prompt or model deploy.**

Knowing you can roll back is different from having actually rehearsed doing it.

05 Observability & Monitoring

Whether you'd actually find out about a quality regression, or only find out when a customer tells you.

- ☐ **You log full input/output pairs for AI responses, not just latency and error rates.**
Traditional APM tells you the request succeeded. It can't tell you the answer was wrong.
 - ☐ **There's a defined metric for "semantic quality," tracked over time, not just uptime.**
A 200 response and a correct response are not the same thing, and your dashboards should distinguish them.
 - ☐ **You have alerting on quality-metric drift, not only on system errors.**
A model that degrades gradually rarely trips an error-rate alert until the damage is already widespread.
 - ☐ **Sampled outputs are reviewed by a human on a regular cadence, not only when something is reported broken.**
Scheduled review catches drift that no automated metric was built to detect.
 - ☐ **You can trace a specific bad output back to the exact prompt, model version, and retrieved context that produced it.**
Without this, every incident investigation starts from scratch.
 - ☐ **Cost and token usage are monitored per-feature, not only in aggregate.**
An aggregate cost spike tells you something changed. Per-feature breakdown tells you what.
 - ☐ **Logged data is handled with the same PII/compliance care as any other production data store.**
Full input/output logging is powerful for debugging and is also a real data-handling liability if treated casually.
-

06 Human-in-the-Loop & Escalation

Whether a human actually gets involved when the system is uncertain — or whether "human in the loop" is a slide, not a system.

- ☐ **Low-confidence outputs are routed to a human review path, not shipped anyway.**
If confidence scoring exists but nothing acts on a low score, it's a metric, not a control.
 - ☐ **There's a clear, tested path for a user to reach a human when the AI can't help.**
An escalation path that exists in documentation but not in the actual product UI doesn't count.
 - ☐ **High-stakes actions (financial, medical, legal, irreversible) require explicit human approval before execution.**
Autonomy should scale with reversibility — the more permanent the action, the less it should run unsupervised.
 - ☐ **Reviewers have the context they need to make a fast, correct call — not just the raw output to approve or reject.**
A reviewer without context becomes a rubber stamp, which defeats the purpose of the review step.
 - ☐ **Human corrections and overrides feed back into your eval set or training data.**
A correction that isn't captured is a lesson the system will need to fail into again before it's learned.
 - ☐ **You've defined and tested what happens when the human reviewer is unavailable or delayed.**
A queue with no timeout or fallback behavior turns a review step into a silent bottleneck.
-

07 Deployment & Change Management

Whether a change to the system is released the way any other production change should be — gradually, and reversibly.

- ☐ **Prompt, model, and pipeline changes go through staging before production.**
Testing a change directly in production is testing on your users.

- ☐ **Changes are rolled out gradually (canary or percentage-based), not all-at-once.**
A gradual rollout turns a bad change into a contained incident instead of a full outage.

- ☐ **You can A/B compare a new prompt or model version against the current one on real traffic before fully switching.**
Eval-set performance and real-traffic performance can diverge in ways only live comparison reveals.

- ☐ **Rollback from a bad deploy takes minutes, not a redeployment cycle.**
If reverting a prompt requires a full CI/CD pipeline run, your recovery time is longer than your users will tolerate.

- ☐ **Feature flags or kill switches exist for AI-powered features specifically, separate from your general feature-flag system.**
An AI feature sometimes needs to be turned off independently of the rest of the product it's embedded in.

- ☐ **Third-party model or API provider changes (deprecations, pricing, behavior updates) are tracked, not discovered by breakage.**
Provider-side changes are outside your control but not outside your responsibility to anticipate.

08 Incident Response & Postmortems

Whether the organization actually learns from an AI failure, or repeats it with a different prompt six months later.

- ☐ **There's a defined process for what happens when an AI failure is reported, not an ad-hoc response each time.**

Who's notified, what gets disabled, and who makes the call should be decided before the incident, not during it.

- ☐ **AI-specific incidents get a postmortem with the same rigor as an infrastructure outage.**

"The model said something wrong" deserves the same root-cause discipline as "the database went down."

- ☐ **Every AI incident produces at least one new eval case or regression test.**

An incident that doesn't leave a permanent test behind is a lesson the system hasn't actually learned.

- ☐ **You have a documented, rehearsed process for notifying affected users when required.**

Figuring out notification obligations for the first time during a live incident costs time you don't have.

- ☐ **Severity levels for AI failures are defined in advance (wrong answer vs. harmful answer vs. data exposure).**

Without pre-agreed severity, every incident becomes a debate about how serious it is before anyone responds.

- ☐ **Postmortem findings are actually reviewed and prioritized, not filed and forgotten.**

A postmortem doc that nobody revisits is documentation of a problem, not progress toward fixing it.

What your gaps are actually telling you

A handful of unchecked boxes scattered across every section is normal — no system does all fifty perfectly, and it isn't meant to be a pass/fail test. What's worth paying attention to is *clustering*: several unchecked items in the same section usually points to a single underlying gap in how that part of the system was built, not fifty unrelated problems.

The most common clusters, in order of how often they show up: no automated groundedness checking (section 3), quality metrics that stop at latency and error rate (section 5), and a human-in-the-loop process that exists on paper but nothing actually routes to it (section 6). If your gaps land in one of those three, you're not unusual — you're early, which is a very different thing.

// IF YOU WANT A SECOND SET OF EYES

Sentinel — an eval-as-MCP-server trust layer

A working reference implementation of several items in sections 1–3: a deterministic-first, judge-escalated scoring layer that catches fabricated citations and prompt-injection compliance before a response ships. Free to clone and run against your own agent.

mleg.tech/sentinel

If your unchecked boxes are spread wide instead of clustered — a handful in most sections rather than a run of them in one — that's usually not a single tool's job to fix. It's closer to a gap in how AI reliability is practiced across the team. That's the more common reason I get brought in directly, to help build the eval and reliability practice itself rather than one piece of it. If that sounds like where you landed, [reach out](#).